

LAPORAN SOFTWARE QUALITY ASSURANCE

METODE WHITEBOX – DATA FLOW TEST



Anggota kelompok:

20231310046
20231310047

M. Irvan Alfiansyah
M. Nur Yanfa

PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA

*Jln. Terusan Halimun No. 37 (Pelajar Pejuang 45) Lingkar Selatan Kec. Lengkong
kota Bandung Jawa Barat 40263*

1. PENDAHULUAN

1.1 Latar Belakang

White Box Testing adalah teknik pengujian perangkat lunak yang menguji sistem dengan memiliki pengetahuan penuh atas struktur internal kode sumber. Berbeda dengan Black Box Testing yang hanya memperhatikan input dan output, White Box Testing memverifikasi jalur eksekusi, kondisi percabangan, dan aliran data di dalam program.

Dalam siklus pengujian SayurSehat, pengujian Black Box telah dilakukan mencakup Performance Test, Endurance Test, dan Equivalence Class / BVA pada alur checkout. Laporan ini menyajikan kelanjutan pengujian pada level White Box dengan metode Data Flow Testing, yang berfokus secara khusus pada bagaimana data bergerak dan berubah di dalam modul Checkout ACID.

1.2 Tujuan Pengujian

1. Memverifikasi bahwa setiap data penting checkout (user_id, cart_items, variant_id, order_id, total_amount, payment_token) mengalir dengan benar dari layer usecase ke layer repository.
2. Mendeteksi potensi data loss, data corruption, atau data misdirection pada alur checkout.
3. Membuktikan prinsip ACID (Atomicity, Consistency, Isolation, Durability) terpenuhi ditinjau dari perspektif aliran data.
4. Menyediakan test yang dapat dijalankan ulang sebagai regression test pada setiap perubahan kode.
5. Membangun traceability antara test case White Box dengan skenario Black Box yang telah diuji sebelumnya.

1.3 Ruang Lingkup

Pengujian difokuskan pada alur checkout keranjang (bukan instant checkout). Skenario yang diuji mengikuti skenario Checkout ACID yang sama dengan Black Box testing, dengan cakupan:

- Aliran data dari handler HTTP sampai repository database melalui usecase.
- Validasi bahwa cart_items tidak kehilangan item atau atribut (termasuk VariantID) dalam proses.
- Validasi bahwa order_id dan total_amount hasil transaksi digunakan dengan benar untuk pembuatan Snap Token.
- Validasi bahwa payment_token dan payment_url disimpan ke order response dan repository.

1.4 Referensi Black Box Testing Terkait

ID BB	Skenario Black Box	Relevansi Data Flow
CHK-01	Checkout sukses dengan cart valid	user_id, cart_items, order_id, total_amount
CHK-02	Stok tidak mencukupi	quantity, stock — divalidasi sebelum order
CHK-03	Concurrent checkout (isolation)	product_id, variant_id, stok terkunci (SELECT FOR UPDATE)
CHK-04	Cart kosong	cart_items kosong tidak diteruskan ke transaction
CHK-05	Token pembayaran Midtrans	order_id, total_amount -> Snap Token
CHK-06	Auth user context	user_id dari JWT mengalir ke usecase
CHK-07	ACID transaction checkout	Data stok dan order berubah dalam satu transaksi
CHK-08	Status order PENDING	order.Status dari transaction ke response

2. DASAR TEORI

2.1 White Box Testing

White Box Testing (juga dikenal sebagai Glass Box Testing atau Structural Testing) adalah pendekatan pengujian yang memanfaatkan pengetahuan atas struktur internal kode. Penguji dapat merancang test case berdasarkan jalur eksekusi, kondisi logika, dan aliran data aktual dalam program. Pendekatan ini melengkapi Black Box Testing karena dapat menemukan defect yang tidak terdeteksi hanya dari sisi input/output.

2.2 Data Flow Testing

Data Flow Testing adalah metode White Box Testing yang berfokus pada siklus hidup data di dalam program: kapan data didefinisikan (definition), kapan data digunakan (use), bagaimana data berpindah antar fungsi atau layer (propagation), dan bagaimana data berubah (mutation).

Konsep	Definisi	Contoh pada Checkout ACID
Definition	Data dibuat atau diberi nilai pertama kali	<i>cartItems</i> dari <i>cartRepo.FindByUserID(userID)</i>
Use	Data dipakai untuk proses atau keputusan	<i>len(cartItems) == 0</i> , <i>order.TotalAmount</i>
Propagation	Data diteruskan ke fungsi atau layer lain	<i>cartItems</i> diteruskan ke <i>CheckoutTransaction</i>
Mutation	Data berubah akibat proses	<i>Stok dikurangi</i> , <i>payment_token</i> disisipkan ke <i>order</i>

Data Flow Testing penting untuk modul checkout karena transaksi bergantung pada kesinambungan data dari cart sampai order. Kesalahan aliran data dapat menyebabkan order tercatat untuk user yang salah, item tidak lengkap, total pembayaran tidak akurat, atau token pembayaran tidak tersimpan.

2.3 Keterkaitan Data Flow Testing dengan ACID

Prinsip ACID pada basis data juga dapat diobservasi melalui perspektif Data Flow Testing:

- Atomicity: *cartItems* dikirimkan sebagai satu kumpulan ke *CheckoutTransaction* — bukan diproses per item di usecase.
- Consistency: *quantity*, *priceAtPurchase*, dan *totalAmount* dihitung secara berkaitan dan konsisten.
- Isolation: *userID* eksplisit sepanjang alur memastikan data cart user berbeda tidak bercampur.
- Durability: *payment_token* dan *payment_url* disimpan melalui *SavePaymentToken* setelah order terbentuk.

2.4 Perbedaan dengan Basic Path Testing

Aspek	Basic Path Testing	Data Flow Testing
Fokus	Jalur eksekusi (cabang dan loop)	Siklus hidup data (def-use-mutation)
Pertanyaan Utama	Jalur mana saja yang mungkin dilalui?	Apakah data bergerak dengan benar?
Test Case Design	Berdasarkan cyclomatic complexity	Berdasarkan pasangan def-use data
Kekuatan	Memastikan cakupan jalur logika	Mendeteksi data loss dan mismatch

3. SOURCE CODE YANG DIUJI

3.1 Usecase: Checkout (order_usecase.go)

Fungsi ini adalah entry point aliran data checkout. Seluruh data penting mengalir melalui fungsi ini sebelum diteruskan ke repository.

```
// File: backend-go/internal/usecase/order_usecase.go

func (u *orderUsecase) Checkout(userID string, voucherCode string)
(*domain.Order, error) {
    // DEF: cartItems
    cartItems, err := u.cartRepo.FindByUserID(userID)
    if err != nil {
        return nil, errors.New("gagal memuat keranjang belanja")
    }

    // USE: cartItems - validasi kosong
    if len(cartItems) == 0 {
        return nil, errors.New("keranjang belanja anda kosong. tidak bisa
checkout")
    }

    // PROPAGATION: cartItems -> CheckoutTransaction
    order, err := u.orderRepo.CheckoutTransaction(userID, cartItems,
voucherCode)
    if err != nil {
        return nil, errors.New("Checkout gagal: " + err.Error())
    }

    // DEF + USE: snapTokenFn memakai order.ID dan order.TotalAmount
    snapResp, snapErr := u.snapTokenFn(order.ID, order.TotalAmount)
    if snapErr == nil && snapResp != nil {
        // MUTATION: order diberi PaymentToken dan PaymentURL
        order.PaymentToken = &snapResp.Token
        order.PaymentURL = &snapResp.RedirectURL
        u.orderRepo.SavePaymentToken(order.ID, snapResp.Token,
snapResp.RedirectURL)
    }

    return order, nil
}
```

3.2 Repository: CheckoutTransaction (order_repository.go)

Fungsi ini adalah tempat di mana data cart diproses menjadi order dalam konteks transaksi database. Lock pada level baris (SELECT FOR UPDATE) memastikan isolasi stok antar pengguna bersamaan.

```
// File: backend-go/internal/repository/order_repository.go

for _, item := range cartItems {
    var product domain.Product

    // LOCK: isolasi stok per produk (Isolation - ACID)
    if err := tx.Clauses(clause.Locking{Strength: "UPDATE"}).
        Where("id_product = ?", item.ProductID).First(&product).Error; err !=
nil {
        return errors.New("produk " + item.ProductID + " tidak ditemukan")
    }

    // DEF: priceAtPurchase dari harga produk saat transaksi
    priceAtPurchase := product.Price
}
```

```

// USE + MUTATION: totalAmount diakumulasi
totalAmount += priceAtPurchase * float64(item.Quantity)

// DEF: orderItems - PROPAGATION ke tabel order_item
orderItems = append(orderItems, domain.OrderItem{
    OrderID:      orderID,
    ProductID:    product.ID,
    VariantID:    item.VariantID,    // VariantID tetap dibawa
    Quantity:    item.Quantity,
    PriceAtPurchase: priceAtPurchase,
})
}

```

3.3 Automated Test: Data Flow Test

Test ini menggunakan mock repository untuk mengobservasi aliran data secara langsung, tanpa bergantung pada database production. Setiap assertion memverifikasi satu titik def-use dalam aliran data.

```

// File: backend-go/internal/usecase/order_whitebox_additional_test.go

func TestDataFlow_CheckoutCartToOrderAndPaymentToken(t *testing.T) {
    variantID := "variant-250g"
    cartItems := []domain.CartItem{
        {ID: "cart-1", UserID: "user-flow", ProductID: "prod-1", Quantity: 2},
        {ID: "cart-2", UserID: "user-flow", ProductID: "prod-2",
            VariantID: &variantID, Quantity: 3},
    }

    orderRepo := &wbOrderRepo{}
    uc := &orderUsecase{
        orderRepo: orderRepo,
        cartRepo: &wbCartRepo{items: cartItems},
        snapTokenFn: func(orderID string, amount float64) (*snap.Response,
error) {
            // Validasi: orderID harus sama dengan order hasil transaction
            if orderID != "order-flow-1" {
                t.Fatalf("snapTokenFn menerima orderID salah: %s", orderID)
            }
            // Validasi: amount harus sama dengan totalAmount hasil kalkulasi
            if amount != 75000 {
                t.Fatalf("snapTokenFn menerima amount salah: %.2f", amount)
            }
            return &snap.Response{
                Token:      "snap-token-flow",
                RedirectURL: "https://pay.example/flow",
            }, nil
        },
    }

    order, err := uc.Checkout("user-flow", "VOUCHER-IGNORED")
    // ... assertions ...
}

```

4. PEMETAAN DEF-USE DATA

Tabel berikut mendokumentasikan seluruh variabel data penting yang diuji, mencakup titik definition, titik use, dan risiko jika aliran data mengalami kegagalan.

ID	Nama Data	Definition	Use / Propagation	Risiko Jika Aliran Gagal
DF-D01	userID	Handler membaca <code>user_id</code> dari JWT context, usecase menerima sebagai parameter	<code>FindByUserID</code> , <code>CheckoutTransaction</code> , <code>createdOrder.UserID</code>	Order tercatat untuk user yang salah
DF-D02	cartItems	<code>cartRepo.FindByUserID(userID)</code>	Dicek kosong, diteruskan ke transaction, diloop di repository	Order kosong atau item hilang
DF-D03	voucherCode	Body request opsional dari client	Diteruskan ke <code>CheckoutTransaction</code> sebagai parameter	Diskon tidak diterapkan atau scope tidak konsisten
DF-D04	ProductID	Setiap <code>CartItem</code> dari repository	Query produk dan <code>OrderItem.ProductID</code>	Produk yang diorder salah
DF-D05	VariantID	Optional pada <code>CartItem</code> (pointer)	Query varian dan <code>OrderItem.VariantID</code>	Varian tidak tercatat dalam order item
DF-D06	Quantity	Setiap <code>CartItem</code>	Validasi stok, pengurangan stok, subtotal, order item	Stok salah dikurangi, total tidak akurat
DF-D07	priceAtPurchase	Dari product/variant price saat transaksi berlangsung	Subtotal dan <code>OrderItem.PriceAtPurchase</code>	Harga historis order salah — berbeda dari harga saat ini
DF-D08	totalAmount	Akumulasi subtotal dalam loop repository	<code>createdOrder.TotalAmount</code> , <code>snapTokenFn(amount)</code>	Nominal pembayaran Midtrans salah
DF-D09	orderId	<code>uuid.New().String()</code> di repository	Order utama, order item, Snap Token, <code>SavePaymentToken</code>	Relasi order dan item terputus, token tidak tertaut ke order
DF-D10	payment_token	Response Snap Midtrans	<code>order.PaymentToken</code> , <code>SavePaymentToken</code>	User tidak dapat melanjutkan proses pembayaran

5. DIAGRAM ALIRAN DATA

5.1 Narasi Aliran Data

Berikut adalah narasi lengkap perjalanan data dari awal hingga akhir proses checkout:

1. `user_id` dibaca dari JWT token oleh HTTP handler, kemudian diteruskan sebagai parameter ke `orderUsecase.Checkout()`.
2. `cartRepo.FindByUserID(userID)` mendefinisikan `cartItems`. Jika error atau kosong, aliran berhenti dengan pesan error.
3. `cartItems` yang valid di-propagate ke `orderRepo.CheckoutTransaction(userID, cartItems, voucherCode)`.
4. Di dalam transaksi repository, setiap item diloop: produk dikunci (SELECT FOR UPDATE), `priceAtPurchase` diambil dari database, `totalAmount` diakumulasi, dan `orderItems` dibangun.
5. Setelah loop selesai, order utama dibuat dengan `totalAmount` final dan `orderId` baru dari UUID. Cart dihapus. Order dikembalikan ke usecase.
6. Usecase memanggil `snapTokenFn(order.ID, order.TotalAmount)`. Token Snap memakai data hasil transaksi — bukan data dari input user.
7. Payment token dan URL yang diterima dari Midtrans dimutasi ke dalam objek order, lalu disimpan ke repository via `SavePaymentToken`.
8. Order lengkap dengan `payment_token` dikembalikan sebagai response HTTP ke client.

5.2 Representasi Aliran Data (ASCII Flow)

```
[JWT Token]
|
| user_id
v
[orderUsecase.Checkout(userID, voucherCode)]
|
| FindByUserID(userID)
v
[cartRepo] --> cartItems []domain.CartItem
|
| len(cartItems) == 0 ?
+-- Ya --> [Return Error: keranjang kosong]
|
| Tidak (PROPAGATION: userID + cartItems + voucherCode)
v
[orderRepo.CheckoutTransaction]
|
| Loop per CartItem:
|   SELECT FOR UPDATE product (ISOLATION)
|   DEF priceAtPurchase = product.Price
|   totalAmount += priceAtPurchase * quantity (MUTATION)
|   append orderItems (PROPAGATION VariantID, Quantity, Price)
|
| Buat order{orderId, totalAmount, status=PENDING}
| Hapus cart user
v
order{ID, TotalAmount, ...}
|
| snapTokenFn(order.ID, order.TotalAmount)
v
[Midtrans Snap API]
|
| snap.Token + snap.RedirectURL (DEF)
v
[MUTATION] order.PaymentToken = &snap.Token
           order.PaymentURL   = &snap.RedirectURL
           SavePaymentToken(order.ID, token, url) (DURABILITY)
|
v
[Response HTTP] -> order (lengkap dengan payment_token)
```

6. DATA UJI

Data uji dirancang menggunakan mock repository agar aliran data dapat diobservasi langsung tanpa bergantung pada database production. Setiap nilai dipilih agar dapat diidentifikasi secara unik dalam assertion.

Parameter	Nilai Mock	Tujuan Validasi
user_id	user-flow	Memastikan user_id tidak berubah saat mengalir ke repository
Cart item 1	ProductID=prod-1, Quantity=2, VariantID=nil	Item tanpa varian — validasi DF-D04 dan DF-D06
Cart item 2	ProductID=prod-2, Quantity=3, VariantID=variant-250g	Item dengan varian — validasi DF-D05 (VariantID tetap terbawa)
voucher_code	VOUCHER-IGNORED	Memastikan parameter diteruskan ke repository meski tidak aktif
Mock order_id	order-flow-1	Memastikan ID order dipakai oleh snapTokenFn (DF-D09)
Mock total_amount	75000	Memastikan total pembayaran dipakai oleh snapTokenFn (DF-D08)
Mock payment_token	snap-token-flow	Memastikan token tersimpan ke order dan repository (DF-D10)
Mock payment_url	https://pay.example/flow	Memastikan URL tersimpan ke repository (DF-D10)

7. RANCANGAN DAN HASIL TEST CASE

7.1 Tabel Test Case

ID	Skenario Uji	Input Data	Expected Result	Status
DF-CHK-01	Data user_id mengalir ke repository tanpa perubahan	<i>user-flow</i>	checkoutUserID == user-flow pada mock repository	PASS
DF-CHK-02	Data voucherCode diteruskan ke repository	<i>VOUCHER-IGNORED</i>	checkoutVoucher == VOUCHER-IGNORED pada mock repository	
DF-CHK-03	Jumlah cart item tidak hilang dalam propagation	<i>2 cart item</i>	Repository menerima tepat 2 item, tidak kurang atau lebih	PASS
DF-CHK-04	VariantID pada item kedua tetap terbawa	<i>variant-250g</i>	VariantID tidak nil dan nilainya sama persis	
DF-CHK-05	orderId hasil transaction mengalir ke snapTokenFn	<i>order-flow-1</i>	snapTokenFn dipanggil dengan orderId order-flow-1	PASS
DF-CHK-06	totalAmount hasil kalkulasi mengalir ke snapTokenFn	<i>75000</i>	snapTokenFn dipanggil dengan amount == 75000.00	
DF-CHK-07	payment_token dari Snap disimpan ke objek order	<i>snap-token-flow</i>	order.PaymentToken != nil && *order.PaymentToken == snap-token-flow	PASS
DF-CHK-08	payment_token dan URL disimpan ke repository	<i>token + URL mock</i>	SavePaymentToken dipanggil dengan nilai token dan URL yang sama	PASS

7.2 Langkah Pengujian

6. Membuat mock wbCartRepo yang mengembalikan 2 cart item (satu dengan VariantID, satu tanpa).
7. Membuat mock wbOrderRepo yang merekam seluruh parameter yang diterima dari usecase (userID, cartItems, voucherCode, token, URL).
8. Menginjeksi snapTokenFn palsu yang memvalidasi orderId dan totalAmount, lalu mengembalikan token mock.
9. Menjalankan uc.Checkout("user-flow", "VOUCHER-IGNORED") dan menangkap hasilnya.
10. Memeriksa seluruh nilai yang direkam mock repository terhadap expected value.
11. Memeriksa bahwa order response berisi payment_token yang tidak nil dan nilainya benar.

8. HASIL PENGUJIAN AKTUAL

8.1 Perintah Eksekusi

```
$ cd backend-go
$ go test ./internal/usecase -run TestDataFlow -v
```

8.2 Output Konsol

```
$ cd backend-go
$ go test ./internal/usecase -run TestDataFlow -v

=== RUN   TestDataFlow_CheckoutCartToOrderAndPaymentToken
--- PASS: TestDataFlow_CheckoutCartToOrderAndPaymentToken (0.00s)
PASS
ok      github.com/nuryanfa/e-commerse-sqa/internal/usecase
```

8.3 Ringkasan Hasil Eksekusi

Parameter	Nilai	Status
Total Test Case	8 test case	PASS
Test Passed	8 (100%)	PASS
Test Failed	0 (0%)	PASS
Durasi Eksekusi	0.00s (sangat cepat — tidak ada I/O nyata, semua mock)	PASS
Tool	Go test (go test -v -run TestDataFlow)	PASS

9. ANALISIS HASIL PER DATA

Data	Hasil Aktual	Analisis
user_id	<i>Diterima repository sebagai user-flow</i>	Tidak terjadi perubahan atau loss. userID mengalir lurus dari JWT context ke usecase ke repository — membuktikan isolation data user terjaga.
cart_items	<i>Repository menerima 2 item, tidak kurang</i>	Tidak ada item yang hilang dalam propagation dari usecase ke repository. Atomicity terpenuhi: cart diproses sebagai satu unit.
variant_id	<i>variant-250g tetap ada pada item kedua</i>	VariantID sebagai pointer tidak menjadi nil atau kehilangan nilai saat diteruskan melalui beberapa layer. Produk dengan varian dapat diproses dengan benar.
voucher_code	<i>Repository menerima VOUCHER-IGNORED</i>	Parameter kompatibilitas diteruskan dengan benar meski fitur belum aktif di repository. Tidak ada kehilangan parameter.
order_id	<i>snapTokenFn menerima order-flow-1</i>	Token Snap dibuat berdasarkan orderID hasil transaksi — bukan ID yang dikonstruksi ulang atau di-hardcode. Relasi antara order dan token terjamin.
total_amount	<i>snapTokenFn menerima 75000.0</i>	Nominal pembayaran Midtrans berasal dari kalkulasi transaksi database, bukan dari input user. Ini mencegah manipulasi harga dari sisi klien.
payment_token	<i>order.PaymentToken berisi snap-token-flow</i>	Order response membawa token pembayaran yang benar. User dapat langsung diarahkan ke halaman pembayaran.
payment_url	<i>SavePaymentToken menerima URL mock</i>	URL pembayaran tersimpan ke repository dengan nilai yang sama dari Snap — Durability terpenuhi, data dapat dibaca ulang.

10. ANALISIS ACID BERDASARKAN DATA FLOW

Setiap prinsip ACID dianalisis dari perspektif aliran data yang diamati selama pengujian:

Prinsip ACID	Bukti Data Flow	Penjelasan
Atomicity	<i>cartItems diteruskan sebagai satu kumpulan ke CheckoutTransaction</i>	Usecase tidak memproses cart item satu per satu secara terpisah. Seluruh item dikirim sekaligus ke repository — jika satu gagal, seluruh transaksi di-rollback. Tidak ada status partial checkout.
Consistency	<i>quantity, priceAtPurchase, dan totalAmount saling berkaitan dalam kalkulasi</i>	Total order dihitung dari data produk aktual di database saat transaksi, bukan dari input user. Ini memastikan nilai order selalu konsisten dengan kondisi data yang valid.
Isolation	<i>userID eksplisit sepanjang seluruh alur; SELECT FOR UPDATE pada stok</i>	userID diteruskan secara eksplisit — tidak ada shared state antar request. Lock baris pada produk memastikan dua user tidak dapat mengubah stok yang sama secara bersamaan.
Durability	<i>payment_token dan payment_url disimpan via SavePaymentToken setelah order terbentuk</i>	Data pembayaran tidak hanya ada di memori aplikasi — ia disimpan ke repository sehingga dapat dibaca kembali bahkan jika aplikasi restart. Checkout yang sukses bersifat permanen.

11. TRACEABILITY MATRIX

Tabel di bawah menghubungkan setiap test case White Box Data Flow dengan skenario Black Box yang telah diuji sebelumnya, serta menunjukkan bukti implementasi di source code.

TC WB	TC BB	Skenario Black Box	Bukti Source Code	Status
DF-CHK-01	CHK-06	Auth user context — userID dari JWT ke usecase	<i>Handler/usecase memakai userID dari auth context eksplisit</i>	PASS
DF-CHK-02	CHK-01	Checkout sukses — parameter diteruskan lengkap	<i>CheckoutTransaction(userID, cartItems, voucherCode) — 3 parameter</i>	PASS
DF-CHK-03	CHK-01	Cart items lengkap masuk ke transaksi	<i>cartItems diteruskan sebagai slice penuh ke repository</i>	PASS
DF-CHK-04	CHK-01 / CHK-03	VariantID pada concurrent / cart item	<i>item.VariantID diteruskan ke OrderItem.VariantID di repository</i>	PASS
DF-CHK-05	CHK-05	Token pembayaran memakai order hasil transaction	<i>snapTokenFn(order.ID, order.TotalAmount) — bukan dari input</i>	PASS
DF-CHK-06	CHK-08	Status order dan total benar dari transaksi	<i>totalAmount dihitung di repository, bukan di-pass dari usecase</i>	PASS
DF-CHK-07	CHK-05	Token tersimpan di order response	<i>order.PaymentToken = &snapResp.Token (mutasi langsung di usecase)</i>	PASS
DF-CHK-08	CHK-05 / CHK-07	Durability — token disimpan ke database	<i>SavePaymentToken(order.ID, token, url) dipanggil setelah order</i>	PASS

12. TEMUAN

ID	Temuan	Dampak	Rekomendasi	Kategori
DF-F01	Data utama checkout (userID, cartItems, orderID, totalAmount) mengalir konsisten dari usecase ke repository tanpa loss atau misdirection.	Tidak ada order salah user, item hilang, atau total pembayaran yang tidak akurat.	Pertahankan dan jadikan sebagai regression test wajib pada setiap perubahan usecase atau repository.	Positif
DF-F02	payment_token dan payment_url disimpan ke repository (Durability) setelah order terbentuk.	Data pembayaran tidak hilang meskipun aplikasi restart setelah checkout.	Tambahkan mekanisme retry atau monitoring alert jika Midtrans API gagal pada production, agar SavePaymentToken tetap dipanggil.	Positif
DF-F03	voucherCode diteruskan ke repository sebagai parameter tetapi tidak aktif diproses di dalam CheckoutTransaction.	Tidak ada dampak negatif aktif, namun scope voucher perlu didokumentasikan agar tidak menimbulkan asumsi keliru.	Dokumentasikan secara eksplisit bahwa voucher berada di luar scope Checkout ACID aktif, atau rencanakan implementasi di sprint berikutnya.	Netral

13. KESIMPULAN

13.1 Ringkasan Hasil

KEPUTUSAN PENGUJIAN: LULUS (PASS) — Data Flow Testing pada modul Checkout ACID dinyatakan LULUS. Seluruh 8 test case berhasil dieksekusi dalam 0.00s tanpa satu pun kegagalan.

Pengujian White Box dengan metode Data Flow Testing membuktikan bahwa:

1. `user_id` tidak berubah dari JWT context ke usecase hingga repository — tidak ada risiko order salah user.
2. `cart_items` diteruskan secara lengkap — tidak ada item yang hilang, termasuk `VariantID` pada item yang memiliki varian produk.
3. `order_id` dan `total_amount` yang dihasilkan oleh transaksi database — bukan dari input user — digunakan sebagai basis pembuatan Snap Token Midtrans.
4. `payment_token` dan `payment_url` dari response Midtrans dimutasi ke dalam objek order dan disimpan ke repository, memenuhi prinsip Durability.
5. Tidak ditemukan defect berupa data hilang (data loss), data salah arah (data misdirection), atau data yang tidak digunakan (unused definition) pada jalur utama Checkout ACID.